

SMS Spam Detection

Отчёт по проекту: SMS Spam Detection

1. Введение и постановка задачи

В проекте решается задача классификации SMS: по тексту нужно понять, относится сообщение к спаму или нет. В данных два класса: `ham` и `spam`.

Основная метрика - `F1` по классу `spam`. Обычных сообщений в корпусе больше, поэтому одной `accuracy` здесь недостаточно: модель может редко предсказывать spam и всё равно выглядеть неплохо. `F1` лучше отражает компромисс между пропущенным спамом и ложными срабатываниями.

2. Поиск и описание данных

Используется SMS Spam Collection: открытый корпус SMS, собранный для исследований фильтрации спама. Датасет доступен в UCI и на Kaggle.

После очистки осталось 5 158 уникальных сообщений:

Класс	Количество
ham	4 516
spam	642

Датасет меньше формального порога в 10 000 строк, но был согласован для проекта. В сыром файле всего две полезные колонки, однако для текстовой классификации это нормально: после TF-IDF одно поле с текстом превращается в набор n-грамм, а поверх него добавляются ручные признаки.

3. Обработка и подготовка данных

Код обработки лежит в `src/data.py` и `src/features.py`.

Сделано:

- загружен архив `sms\_spam\_collection.zip`;
- оставлены поля `label` и `message`;
- метки закодированы как `ham = 0`, `spam = 1`;

- удалены пустые сообщения и точные дубли;
- добавлены признаки: длина сообщения, число слов, цифр, uppercase-доля, пунктуация, наличие URL, телефона, валюты и СТА-слов.

Данные делятся стратифицированно в пропорции `70/15/15`: train, validation, test.
`CountVectorizer` и `TfidfVectorizer` находятся внутри sklearn pipeline и обучаются только на train-части, поэтому validation/test не участвуют в построении словаря и IDF.

Выводы из EDA:

- классы несбалансированы: spam составляет меньшую часть корпуса, поэтому F1 по spam важнее общей accuracy;
- spam-сообщения чаще содержат цифры, телефоны, денежные маркеры и СТА-слова;
- длина сообщения сама по себе не решает задачу, но помогает как дополнительный слабый сигнал.

Графики лежат в `report/images/`.

4. Baseline-модель

Первый baseline - `CountVectorizer + MultinomialNB`. Это простая связка для текстовой классификации без ручных признаков.

На test она дала:

Модель	Accuracy	Precision spam	Recall spam	F1 spam
CountVectorizer + MultinomialNB	0.979	0.966	0.866	0.913

Результат baseline получился высоким, потому что часть spam-сообщений содержит довольно явные слова и шаблоны: `free`, `call`, `claim`, короткие номера и т.п.

5. Эксперименты

В экспериментах сравнивались простые текстовые модели и вариант с ручными признаками. Идея была проверить, хватает ли n-грамм для коротких SMS и дают ли пользу признаки вроде телефона, валюты и СТА-слов.

Гипотеза	Как проверялось	Результат
----------	-----------------	-----------

|---|---|---|

| Bag-of-words достаточно для первого baseline | `CountVectorizer + MultinomialNB` |

`F1\_spam = 0.913` на test |

| TF-IDF даст более устойчивые признаки | `TF-IDF + LogisticRegression` | `F1\_spam = 0.819` на test, recall оказался ниже |

| Margin-based линейная модель лучше разделит классы | `TF-IDF + LinearSVC` | `F1\_spam = 0.951` на validation |

| Ручные признаки усилят линейную модель | `TF-IDF + manual features + LogisticRegression` |

`F1\_spam = 0.928` на test |

Итоговая таблица:

Модель	Split	Accuracy	Precision spam	Recall spam	F1 spam
CountVectorizer + MultinomialNB	test	0.979	0.966	0.866	0.913
TF-IDF + LogisticRegression	test	0.961	0.986	0.701	0.819
TF-IDF + LinearSVC	val	0.988	1.000	0.906	0.951
TF-IDF + LinearSVC	test	0.978	0.955	0.866	0.908
TF-IDF + ручные признаки + LogisticRegression	test	0.983	1.000	0.866	0.928

6. Финальная модель и интерпретируемость

Для деплоя выбрана `TF-IDF + ручные признаки + LogisticRegression`. `LinearSVC` был сильнее на validation, но логистическая регрессия удобнее для интерфейса: она даёт вероятность spam-класса через `predict\_proba`, и это значение выводится как `spam\_score`.

Интерпретация признаков:

- n-граммы вроде `free`, `call`, `claim`, `prize`, `urgent` повышают вероятность spam;
- наличие телефона, валюты и СТА-слов работает как дополнительный сигнал;
- обычные бытовые формулировки без цифр, ссылок и призывов к действию чаще классифицируются как `ham`.

7. Деплой

Для CP3 добавлены два локальных способа использовать модель.

FastAPI:

- `GET /health` - проверка состояния;
- `POST /predict` - предсказание по JSON с полем `message`.

Пример:

```
```json
{
 "message": "URGENT! You won a free prize. Call now to claim cash."
}
```
```

Ответ:

```
```json
{
 "label": "spam",
 "target": 1,
 "spam_score": 0.9902
}
```
```

Streamlit:

- текстовое поле для SMS;
- кнопка проверки;
- вывод класса и `spam\_score`;
- два встроенных примера: обычное сообщение и сообщение, похожее на спам.

Изображения деплоя:

- `report/images/api\_predict\_example.png`;
- `report/images/streamlit\_interface\_example.png`.

Демонстрация: https://disk.yandex.ru/i/pJhz8z5Rq_DAw

Локально API запускается командой `uvicorn app.api:app --host 127.0.0.1 --port 8000`,
интерфейс - командой `streamlit run app/streamlit_app.py --server.address 127.0.0.1
--server.port 8501`. Скриншоты работы сохранены в `report/images/`.

8. Заключение и выводы

Проект можно запустить как локальный сервис. API подходит для программного вызова модели, Streamlit - для ручной проверки одного сообщения.

Главное ограничение - размер и возраст датасета. Для реального применения стоило бы собрать более свежие сообщения, проверить модель на новых данных и отдельно подобрать порог `spam\_score`, чтобы контролировать ложные срабатывания.